

Task 1 : Unique Launch Sites

Identifying Primary SpaceX Launch Hubs

EXECUTED SQL QUERY:

```
SELECT DISTINCT "Launch_Site" FROM SPACEXTBL;
```

QUERYRESULT OUTPUT

[4] : **Launch_Site**

CCAFS LC-40

VAFB SLC-4E

KSC LC-39A

CCAFS SLC-40

Task 2: Cape Canaveral Launch Records

Filtering Initial CCAFS Missions

EXECUTED SQL QUERY:

```
SELECT * FROM SPACEXTBL WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
```

[6]:

	Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
0	2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
1	2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of...	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2	2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
3	2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
4	2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Task 3 & 4: Payload Mass Analysis

Aggregating Payload Metrics by Customer & Booster

Task 3: Total Payload for NASA (CRS)

```
SELECT SUM("PAYLOAD_MASS__KG_") FROM SPACEXTBL
WHERE "Customer" = 'NASA (CRS)';
```

45,596 kg

Total payload mass delivered across all NASA Commercial Resupply Services missions.

Task 4: Average Payload for Booster F9 v1.1

```
SELECT AVG("PAYLOAD_MASS__KG_") FROM SPACEXTBL
WHERE "Booster_Version" = 'F9 v1.1';
```

2,928.4 kg

Mean payload capacity carried by the Falcon 9 v1.1 variant.

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
[7]: pd.read_sql('SELECT SUM("PAYLOAD_MASS__KG_") FROM SPACEXTBL WHERE "Customer" = "NASA (CRS)"', con)
```

SUM("PAYLOAD_MASS__KG_")	
0	45596

Task 4

Display average payload mass carried by booster version F9 v1.1

```
[15]: pd.read_sql('SELECT AVG("PAYLOAD_MASS__KG_") FROM SPACEXTBL WHERE "Booster_Version" = "F9 v1.1"', con)
```

AVG("PAYLOAD_MASS__KG_")	
0	2928.4

Task 5 & 6: Landing Milestones & Payload Filtering

Ground Pad First Success & Drone Ship Performance

Task 5: First Ground Pad Landing Success

```
SELECT MIN("Date") FROM SPACEXTBL WHERE  
"Landing_Outcome" = 'Success (ground pad)';
```

2015-12-22

Historic milestone marking SpaceX's first successful land recovery of a orbital booster.

Task 6: Drone Ship Success (4,000 - 6,000 kg)

```
SELECT "Booster_Version" FROM SPACEXTBL WHERE  
"Landing_Outcome" = 'Success (drone ship)' AND  
"PAYLOAD_MASS__KG_" BETWEEN 4000 AND 6000;
```

4 Boosters

Boosters: F9 FT B1022, F9 FT B1026, F9 FT B1021.2, F9 FT B1031.2

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
16]: pd.read_sql('SELECT MIN("Date") FROM SPACEXTBL WHERE "Landing_Outcome" = "Success (ground pad)"', con)
```

```
16]:
```

	MIN("Date")
--	-------------

0	2015-12-22
---	------------

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
17]: pd.read_sql('SELECT "Booster_Version" FROM SPACEXTBL WHERE "Landing_Outcome" = "Success (drone ship)" AND "PAYLOAD_MASS__KG_" BETWEEN 4000 AND 6000', con)
```

```
17]:
```

	Booster_Version
--	-----------------

0	F9 FT B1022
---	-------------

1	F9 FT B1026
---	-------------

2	F9 FT B1021.2
---	---------------

3	F9 FT B1031.2
---	---------------

Task 7: Mission Outcome Breakdown

Evaluating Overall Flight Success Rate

EXECUTED SQL QUERY:

```
SELECT "Mission_Outcome" COUNT(*) FROM SPACEXTBL GROUP BY "Mission_Outcome";
```

QUERYRESULT OUTPUT

Success: 98 missions
Success (with minor variants): 2 missions
Failure (in flight): 1 mission (CRS-7 launch failure)

Task 7

```
pd.read_sql("SELECT "Mission_Outcome", COUNT(*) FROM SPACEXTBL GROUP BY "Mission_Outcome", con)
```

List the total number of successful and failure mission outcomes

```
[19]: pd.read_sql("SELECT "Mission_Outcome", COUNT(*) FROM SPACEXTBL GROUP BY "Mission_Outcome", con)
```

```
[19]:
```

	Mission_Outcome	COUNT(*)
0	Failure (in flight)	1
1	Success	98
2	Success	1
3	Success (payload status unclear)	1

Task 8: Maximum Payload Carrying Boosters

Subquery Analysis for Heavy Payload Missions (15,600 kg)

EXECUTED SQL QUERY:

```
SELECT DISTINCT "Booster_Version" FROM SPACEXTBL WHERE "PAYLOAD_MASS_KG_" = (SELECT  
MAX("PAYLOAD_MASS_KG_") FROM SPACEXTBL);
```

QUERYRESULT OUTPUT

Task 8

```
pd.read_sql('SELECT DISTINCT "Booster_Version" FROM SPACEXTBL WHERE "PAYLOAD_MASS_KG_" = (SELECT MAX("PAYLOAD_MASS_KG_") FROM  
SPACEXTBL)', con)
```

List all the booster versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
[18]: pd.read_sql('SELECT DISTINCT "Booster_Version" FROM SPACEXTBL WHERE "PAYLOAD_MASS_KG_" = (SELECT MAX("PAYLOAD_MASS_KG_") FROM SPACEXTBL)', con)
```

```
[18]:
```

	Booster_Version
--	-----------------

0	F9 B5 B1048.4
---	---------------

1	F9 B5 B1049.4
---	---------------

2	F9 B5 B1051.3
---	---------------

3	F9 B5 B1056.4
---	---------------

4	F9 B5 B1048.5
---	---------------

5	F9 B5 B1051.4
---	---------------

6	F9 B5 B1049.5
---	---------------

Task 9: Drone Ship Landing Failures in 2015

Date & Pattern Handling in SQLite

EXECUTED SQL QUERY:

```
SELECT SUBSTR("Date", 6, 2) AS Month, "Landing_Outcome", "Booster_Version", "Launch_Site" FROM  
SPACEXTBL WHERE SUBSTR("Date", 1, 4) = '2015' AND "Landing_Outcome" = 'Failure (drone ship)';
```

QUERYRESULT OUTPUT

Month 01 (Jan 2015): F9 v1.1 B1012 | CCAFS LC-40 | Failure (drone ship)

Month 04 (Apr 2015): F9 v1.1 B1015 | CCAFS LC-40 | Failure (drone ship)

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

```
20]: pd.read_sql('SELECT SUBSTR("Date", 6, 2) AS Month, "Landing_Outcome", "Booster_Version", "Launch_Site" FROM SPACEXTBL WHERE SUBSTR("Date", 1, 4) = '2015' AND "Landing_Outcome" = 'Failure (drone ship)')'
```

```
20]:
```

	Month	Landing_Outcome	Booster_Version	Launch_Site
0	01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
1	04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Task 10

Task 10: Landing Outcome Ranking (2010–2017)

Historical Distribution of First Stage Recoveries

EXECUTED SQL QUERY:

```
SELECT "Landing_Outcome", COUNT(*) AS Count FROM SPACEXTBL WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY "Landing_Outcome" ORDER BY Count DESC;
```

QUERYRESULT OUTPUT

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
[21]: pd.read_sql('SELECT "Landing_Outcome", COUNT(*) AS Count FROM SPACEXTBL WHERE "Date" BETWEEN "2010-06-04" AND "2017-03-20" GROUP BY "Landing_Outcome" ORDER BY Count DESC')
```

```
[21]:
```

	Landing_Outcome	Count
0	No attempt	10
1	Success (drone ship)	5
2	Failure (drone ship)	5
3	Success (ground pad)	3
4	Controlled (ocean)	3
5	Uncontrolled (ocean)	2
6	Failure (parachute)	2
7	Precluded (drone ship)	1

Web Scrapping Architecture: Wikipedia Data

Automated HTML Table Parsing with BeautifulSoup

TASK 1: Request the Falcon9 Launch Wiki page from its URL

First, let's perform an HTTP GET method to request the Falcon9 Launch HTML page, as an HTTP response.



```
[3]: import requests
from bs4 import BeautifulSoup

# Define static_url and headers
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.0.0 Safari/537.36'
}

# use requests.get() method with the provided static_url and headers
# assign the response to a object
response = requests.get(static_url, headers=headers)
soup = BeautifulSoup(response.text, 'html.parser')
print(soup.title)
```

<title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>

Create a BeautifulSoup object from the HTML response

```
[5]: # Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(response.text, 'html.parser')
```

Print the page title to verify if the BeautifulSoup object was created properly

Web Scrapping Architecture: Wikipedia Data

Automated HTML Table Parsing with BeautifulSoup

TASK 2: Extract all column/variable names from the HTML table header

Next, we want to collect all relevant column names from the HTML table header



Let's try to find all tables on the wiki page first. If you need to refresh your memory about BeautifulSoup, please check the external reference link towards the end of this lab

```
[9]: # Use the find_all function in the BeautifulSoup object, with element type `table`  
# Assign the result to a list called `html_tables`  
response = requests.get(static_url, headers=headers)  
soup = BeautifulSoup(response.text, 'html.parser')  
soup.title  
html_tables = soup.find_all('table')
```

Starting from the third table is our target table contains the actual launch records.

```
[10]: # Let's print the third table and check its content  
first_launch_table = html_tables[2]  
print(first_launch_table)
```

```
<table class="wikitable plainrowheaders collapsible" id="mwmQ" style="width: 100%;">  
<tbody id="mwmQ"><tr id="mwmw"><th id="mwnA" scope="col">Flight No.</th>  
<th id="mwnQ" scope="col">Date and<br id="mwng"/>time (<a href="https://en.wikipedia.org/wiki/Coordinated_Universal_Time" id="mwnw" r  
el="mw:WikiLink" title="Coordinated Universal Time">UTC</a></th>  
<th id="mwoA" scope="col"><a href="https://en.wikipedia.org/wiki/List_of_Falcon_9_first-stage_boosters" id="mwoQ" rel="mw:WikiLink" t  
itle="List of Falcon 9 first-stage boosters">Version,<br id="mwog"/>Booster</a> <sup about="#mwt70" class="mw-ref reference" data-mw  
='{"name":"ref","attrs":{"name":"booster","group":"lower-alpha"},"body":{"id":"mw-reference-text-cite_note-booster-11"},"parts":[{"te  
mplate":{"target":{"wt":"efn","href":"./Template:Efn"},"params":{"name":{"wt":"booster"},"1":{"wt":"Falcon 9 first-stage boosters are  
designated with a construction serial number and an optional flight number when reused, e.g. B1021.1 and B1021.2 represent the two fl  
ights of booster [[B1021]]. Launches using reused boosters are denoted with a recycled symbol ♻️."}},"i":0}}}]' id="cite_ref-booster_
```

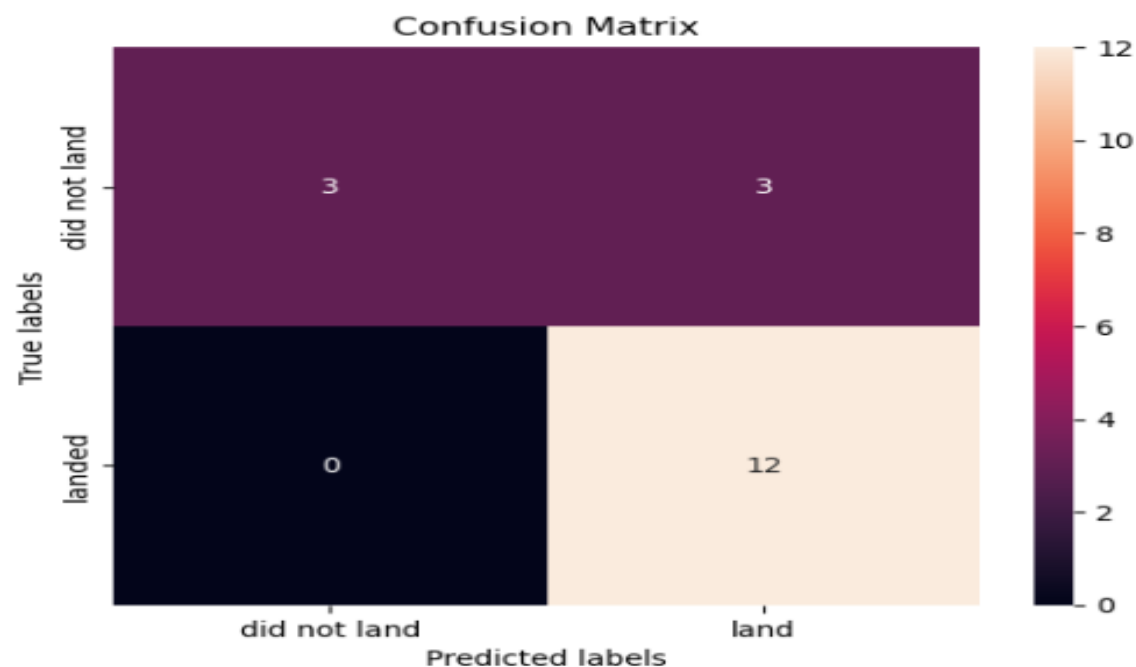
Interactive Data Analytics (Maps & Dashboards) & Machine Learning Landing Prediction Model

Next Steps for Binary Classification

```
# 4. Define parameters and fit model
parameters = {'C': [0.01, 0.1, 1],
              'penalty': ['l2'],
              'solver': ['lbfgs']}

lr = LogisticRegression()
logreg_cv = GridSearchCV(lr, parameters, cv=10)
logreg_cv.fit(X_train, Y_train)

# 5. Make predictions and display matrix
yhat = logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test, yhat)
```



Load the dataframe

Load the data

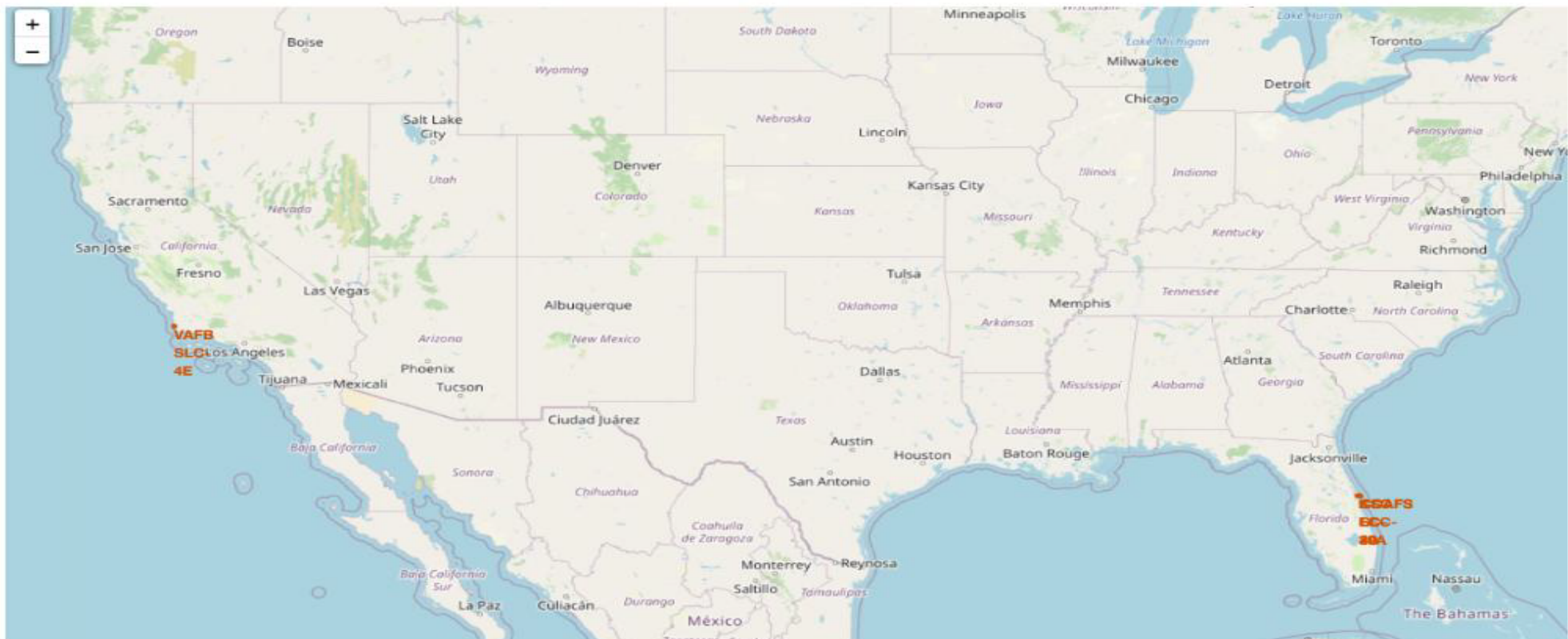
Interactive Data Analytics (Maps & Dashboards) & Machine Learning Landing Prediction Model

Next Steps for Binary Classification

```
folium.map.Marker(coordinate, icon=DivIcon(icon_size=(20,20),icon_anchor=(0,0), html='<div style="font-size: 12; color:#d35400;">  
<b>%s</b></div>' % 'label', ))
```

```
[ ]: # Initial the map  
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)  
# For each launch site, add a Circle object based on its coordinate (Lat, Long) values. In addition, add Launch site name as a popup label
```

The generated map with marked launch sites should look similar to the following:



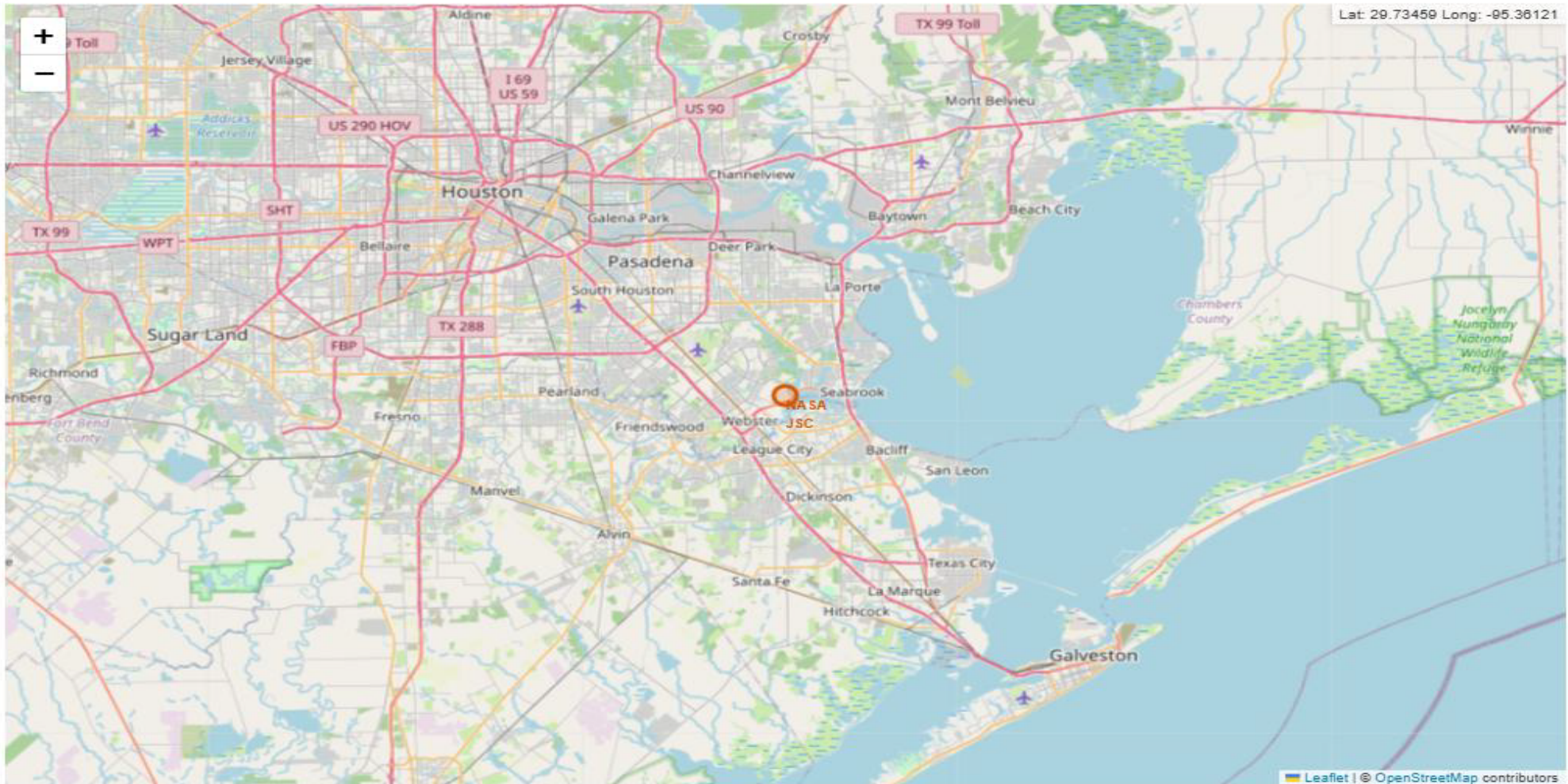
Now, you can explore the map by zoom-in/out the marked areas , and try to answer the following questions:

Interactive Data Analytics (Maps & Dashboards) & Machine Learning Landing Prediction Model

Next Steps for Binary Classification

```
site_map.add_child(mouse_position)  
site_map
```

[13]:



Now zoom in to a launch site and explore its proximity to see if you can easily find any railway, highway, coastline, etc. Move your mouse to these points and mark down their coordinates (shown on the top-left) in order to the distance to the launch site.

Data collection
